Week 01

AI 협업 코딩과 Colab 시작

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

오늘의 목표

1. Colab에서 셀을 실행하고 **실행 순서**의 의미를 이해한다
2. 변수 · 자료형 · f-string으로 출력 프로그램을 만든다
3. 오류 메시지를 **마지막 줄부터** 읽고 고친다
4. 좋은 프롬프트로 AI에게 묻고, 받은 코드를 **실행해서 검증**한다

> **오늘의 산출물**: 실행 가능한 첫 Colab 노트북 + AI 활용 기록

이 수업은 어떻게 진행되나요?

구간	시간	활동
도입 · 개념	20분	핵심 개념과 시범
안내 실습	45분	`practice.ipynb`를 함께 실행
독립 · 짝 실습	35분	직접 해보기, AI 협업
공유 · 점검	20분	오류 사례 공유, 과제 안내

이번 주 Colab 실습 열기

15주 로드맵

구간	주차	주제
Python 기초 다지기	1~5	Colab · 문자열 · 제어문 · 자료구조 · 함수
프로그램 구조화	6~8	파일 · 예외 · 클래스 · **중간 실기**
데이터 분석	9~13	NumPy · pandas · 정제 · EDA · 시각화
품질과 발표	14~15	AI 디버깅 · 테스트 · **최종 발표**

매주 **실습 노트북 → 과제 노트북(3~4문제) → AI 활용 기록**을 반복합니다.

AI는 정답 생성기가 아니라 학습 파트너

정답 생성기로 쓸 때	학습 파트너로 쓸 때
코드를 그대로 복사해 제출	코드를 읽고 실행해서 확인
오류가 나면 다시 물어봄	오류 메시지를 먼저 읽고 가설을 세움
왜 되는지 모름	한 줄씩 설명할 수 있음
시험에서 막힘	AI 없이도 핵심은 직접 작성

> 최종 결과에 대한 책임은 **AI가 아니라 제출한 사람**에게 있습니다.

Colab 첫걸음

- **코드 셀**: `Shift + Enter`로 실행
- **텍스트 셀**: 설명(Markdown)
- 셀 왼쪽 `[3]` → **3번째로 실행된 셀**

단축키	기능
`Ctrl + M B`	아래에 셀 추가
`Ctrl + /`	주석 처리/해제
런타임 → 다시 시작 및 모두 실행	처음부터 다시 실행

실행 순서가 결과를 바꾼다

```
count = 0            # 셀 A
```

```
count = count + 1    # 셀 B
print(count)
```

- A → B → B → B 순서로 실행하면? **3**
- 그다음 B만 한 번 더? **4**
- 위에서 아래로 읽은 순서가 아니라 **실행한 순서**가 기억됩니다

> 결과가 이상하면 **런타임 → 다시 시작 및 모두 실행**

변수와 자료형

```
name = "이영호"          # str   문자열
year = 2                 # int   정수
height_m = 1.75          # float 실수
is_enrolled = True       # bool  참/거짓
print(type(year))        # <class 'int'>
```

- `=`는 "같다"가 아니라 **오른쪽 값을 왼쪽 이름에 저장**
- 변수 이름은 의미가 드러나게: `a` 보다 `total_hours`

❓ 예상해 보기

```
print("3" + "4")
print(3 + 4)
print(int("3") + 4)
print("학년: " + str(2))
```

정답: `34` / `7` / `7` / `학년: 2`

- 글자 + 글자 = **연결**, 숫자 + 숫자 = **덧셈**
- 형 변환: `int()` · `float()` · `str()`

f-string: 값을 문장 속에

```
major = "컴퓨터공학과"
print(f"{name}님은 {major} {year}학년입니다.")
print(f"내년에는 {year + 1}학년이 됩니다.")
```

서식	결과
`f"{1500000:,}"`	`1,500,000`
`f"{3.14159:.2f}"`	`3.14`
`f"{0.25:.0%}"`	`25%`
`f"{'A':>5}"`	`    A` (오른쪽 정렬)

`input()`은 항상 문자열

```
birth = input("태어난 해: ")   # "2005"  ← 문자열!
age = 2026 - int(birth)         # 숫자로 바꿔야 계산 가능
```

이 수업의 노트북은 자동 검증을 위해 이렇게 씁니다.

```
USE_INPUT = False
if USE_INPUT:
    birth = input("태어난 해: ")
else:
    birth = "2005"      # 미리 정한 테스트 값
```

오류 메시지 읽는 법

```
TypeError                          Traceback (most recent call last)
----> 2 age = 2026 - birth
TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

1. **마지막 줄** → `TypeError: int와 str은 뺄 수 없음`
2. **화살표** → 2번째 줄
3. **가설** → birth가 문자열이구나
4. **수정** → `int(birth)`

자주 만나는 오류 5가지

오류	흔한 원인	예
`NameError`	오타, 실행 안 한 셀	`print(studnet)`
`TypeError`	자료형 불일치	`"3" + 4`
`SyntaxError`	괄호·따옴표·콜론 누락	`print("안녕"`
`ValueError`	변환 불가능한 값	`int("삼")`
`IndentationError`	들여쓰기 불일치	`if` 다음 줄

> 오류는 실패가 아니라 **Python이 주는 힌트**입니다

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 환경 진단 셀 실행 (Python 버전, Colab 여부)
- 자료형 확인, `"3" + "4"` 예상해 보기

Level 2 바꾸기

- 자기소개 카드의 값 **두 곳 이상** 바꾸기
- 3시간 수업일 때 총 시간 출력 `총 45시간 (2,700분)`

📓 노트북 `1. 따라하기` ~ `3. 바꾸기`

실습 3 (Level 3): 수강 준비 진단

```
def diagnose(has_google, has_github, can_use_ai, ran_colab):
    missing = ""
    if not has_github:
        missing += "GitHub 계정 "
    ...
    if missing == "":
        return "✅ 수강 준비 완료"
    return f"⚠️ 준비 필요: {missing.strip()}"
```

`True + True + False` → **2** (참은 1, 거짓은 0)

한 번 성공했다고 끝이 아니다

```
print(diagnose(True, True, True, True))       # 모두 준비
print(diagnose(False, False, False, False))   # 모두 미준비
print(diagnose(True, False, True, True))      # GitHub만 없음
```

`assert`로 기대 결과를 코드로 남기기

```
assert diagnose(True, True, True, True) == "✅ 수강 준비 완료"
assert "GitHub 계정" in diagnose(True, False, True, True)
```

조건이 거짓이면 `AssertionError`로 **멈춥니다**

좋은 프롬프트의 4요소

요소	예시
역할 · 맥락	Python을 배우는 대학생, Colab 사용
요구사항	태어난 해(문자열)로 2026년 기준 나이를 돌려주는 함수
제약 조건	함수와 f-string만, 외부 라이브러리 금지
출력 형식	코드 + 한국어 주석 + 테스트 예시 3개

❌ `나이 계산 코드 줘`

✅ 위 4요소를 모두 담은 3~5줄 프롬프트

🤖 AI 코드가 틀릴 때

"나이 계산 코드 줘" → AI의 답

```
birth = input("태어난 해: ")
age = 2026 - birth
print("나이는 " + age + "살")
```

- 버그 1: `2026 - "2005"` → `TypeError`
- 버그 2: `"나이는 " + 21` → `TypeError`
- 원인: 프롬프트에 **입력 자료형과 출력 형식**이 없었음

고친 코드와 검증

```
def calc_age(birth_year_text, base_year=2026):
    birth_year = int(birth_year_text.strip())
    return base_year - birth_year

calc_age("2005")      # 정상 → 21
calc_age(" 2026 ")    # 경계: 공백, 올해 출생 → 0
calc_age("이천오")    # 잘못된 입력 → ValueError (6주차에 처리)
```

정상 · 경계 · 잘못된 입력 **세 가지**로 검증하는 습관

AI 코드 검증 루틴 (매주 사용)

1. **읽기** — 한 줄씩 읽고 모르는 부분 표시
2. **예상** — 실행 전 결과 예상
3. **실행** — 정상 · 경계 · 잘못된 입력
4. **수정** — 다르면 원인을 찾아 직접 수정
5. **기록** — 프롬프트, 채택, 수정과 이유

> 이 다섯 단계가 매주 과제의 **AI 활용 기록**이 됩니다

AI 활용 기록 양식

항목	예시
사용한 AI 도구	ChatGPT, Gemini, Claude 등
내가 쓴 프롬프트	(그대로 붙여 넣기)
채택한 부분	`"\n".join()`으로 줄 합치기
직접 고친 부분과 이유	`print` → `return` (자동 확인 실패)
검증 입력	정상 1, 경계 1, 잘못된 입력 1

AI를 쓰지 않았다면 **"사용 안 함"** 이라고 적습니다

AI 활용 원칙

- AI가 만든 코드는 **직접 실행하고 설명할 수 있을 때만** 제출
- 개인정보 · 비밀번호 · API 키를 프롬프트에 넣지 않기
- 중간 실기 등 **AI 사용 범위가 정해진 평가**는 안내를 따름
- 다른 학생의 코드나 AI 답을 공유해 그대로 제출하면 부정행위
- 모르는 개념은 "설명해 줘 → 예시 → 내가 다시 설명" 순서로

이번 주 과제 (10점)

문제	내용	배점
1	자기소개 변수 4개 (자료형 조건)	2
2	온도 변환 함수 `c_to_f`, `f_to_c`	2
3	버그 3개 고치기 + 버그 기록표	3
4	AI와 함께 `make_card` 만들기 + AI 활용 기록	3

- `assignment.ipynb`를 **드라이브에 사본 저장** 후 작성
- 자동 확인 ✅ + AI 활용 기록 + 성찰 → `.ipynb` 제출

자기 점검

1. 같은 셀을 두 번 실행하면 결과가 달라질 수 있는 이유는?
2. `input()`으로 받은 `"20"`에 1을 더하려면?
3. `print("점수: " + 90)`에서 나는 오류의 이름과 해결법은?
4. 좋은 프롬프트의 4요소는?
5. AI가 준 코드를 검증할 때 사용할 입력 세 종류는?

정리와 다음 주

오늘	핵심
Colab	실행 순서 · 다시 시작 및 모두 실행
자료형	`int` · `float` · `str` · `bool`, 형 변환
출력	f-string 서식 `:,` `:.2f` `:>8`
오류	마지막 줄 → 종류 → 위치 → 수정
AI	4요소 프롬프트 → 검증 → 수정 → 기록

다음 주 · Week 02 입력 · 계산 · 문자열 — 여행 경비 계산기